

11.模型转换工具rknn_convert

1. rknn_convert 简介

rknn_convert 是RKNN-Toolkit2提供的一套常用模型转换工具，通过封装API接口，**用户只需编辑模型对应的 yml 配置文件**，就可以通过**指令**转换模型。以下是如何使用 rknn_convert 工具的示例命令以及支持的指令参数：

代码块

```
1 python -m rknn.api.rknn_convert -t rk3588 -i ./model_config.yml -o  
  ./output_path
```

通过使用上述命令和参数，用户可以将模型转换为RKNN格式，并将转换后的模型保存到指定的输出路径。支持的指令参数说明如下：

- -m: 是 Python 解释器的一个标准命令行参数，作用是将一个 Python 模块（module）作为脚本直接运行。
- -t: target_platform，目标平台可以选择rv1103, rv1103b,rv1106, rv1106b, rv1126b, rk2118, rk3562, rk3566, rk3568, rk3576或rk3588。
- -i: 模型配置文件（.yml）路径。
- -o: 转换后模型输出路径。
- -e: (选填) 评估连板运行时model的耗时和内存占用，若开启请输入-e。注：一定要连接相应开发板并正确设置 target_platform，否则会报错，当有多设备时可通过 -d 参数指定设备ID。
- -a: (选填)评估生成的rknn模型精度，开启模拟器精度评估请输入-a "xx1.jpg xx2.jpg"，若要开启连板精度评估请配合-d参数使用。
- -v: (选填)指定是否要在屏幕上打印详细日志信息，若开启打印模式请输入-v。
- -d: (选填)单个adb设备使用-d，多adb设备使用-d device_id，device_id通过adb devices查询。

模型转换配置详见下表：

参数名	填写内容
-name	模型输出名称
-platform	原始模型使用的框架，支持tensorflow、tflite、caffe、onnx、pytorch、darknet
-model_file_path	原始模型文件路径，适用于单模型文件输入，例：tensorflow、tflite、onnx、pytorch
-quantize	是否开启量化
-dataset	量化dataset文件路径（相对yml配置文件路径），若要开启accuracy_analysis此项必填
-prototxt_file_path	platform为caffe时，模型的prototxt文件
-caffemodel_file_path	platform为caffe时，模型的caffemodel文件
-darknet_cfg_path	platform为darknet时，模型的cfg文件
-darknet_weights_path	platform为darknet时，模型的weight文件
-subgraphs	描述输入输出shape等信息。除特定框架外，一般情况下该参数及附带的子参数可不写，使用模型默认值
---input_size_list(子参数)	输入tensor的shape
---inputs(子参数)	输入tensor的名称
---outputs(子参数)	输出tensor的名称
-configs	对应rknn.config()配置
---quantized_dtype(子参数)	量化类型，RKNN_toolkit2: 可填写 [asymmetric_quantized-8]，不输入用默认值
---mean_values(子参数)	输入的均值归一数，模型为单输入RGB如[123.675,116.28,103.53]，若为多输入如[[123,116,103],[255,255,255]]

参数名	填写内容
---std_values(子参数)	输入的方差归一数，模型为单输入RGB如[58.395,58.295,58.391]，若为多输入如[[127,127,127],[255,255,255]]
---quant_img_RGB2BGR(子参数)	用于控制量化时加载量化校正图像时是否需要先进行RGB到BGR的转换，默认值是False
---quantized_algorithm(子参数)	量化算法，可选['normal', 'kl_divergence', 'mmse']，默认为normal
---quantized_method(子参数)	量化方式，RKNN_toolkit2可选['layer', 'channel']，默认为channel
---optimization_level(子参数)	设置优化级别。默认为3，表示使用所有默认优化选项
---model_pruning(子参数)	修剪模型以减小模型大小，默认为false，开启为true
---quantize_weight(子参数)	当quantize参数为false时，通过量化一些权重来减小rknn模型的大小。默认为false，开启为true
---single_core_model(子参数)	是否仅生成单核模型，可以减小RKNN模型的大小和内存消耗。默认值为False。目前对RK3588/RK3576生效。默认值为False

2. model_config.yml

代码块

```

1  models:
2      name: resnet_18                # 模型输出名称
3      platform: pytorch              # 原始模型使用的框架
4      model_file_path: ./resnet18.pt # 原模型路径
5      subgraphs:                     # 描述输入输出shape等信息
6          input_size_list:
7              - 1, 3, 224, 224
8      quantize: true                  # 是否量化
9      dataset: ./dataset.txt         # 量化dataset文件路径（相对yml路径）
10     configs:
11         quantized_dtype: asymmetric_quantized-8 # 量化类型
12         mean_values: [123.675, 116.28, 103.53] # rknn.config的mean_values参数
13         std_values: [58.395, 58.395, 58.395]   # rknn.config的std_values参数
14         quant_img_RGB2BGR: false                # 不进行RGB2BGR转换
15         quantized_algorithm: normal             # 量化算法
16         quantized_method: channel               # 量化方法
17

```

3. rknn_convert 代码测试

02_resnet18_convert

使用resnet18的demo进行一下测试。输入如下命令：

代码块

```
1 python -m rknn.api.rknn_convert -t rk3588 -i ./model_config.yml -o
  ./output_path
```

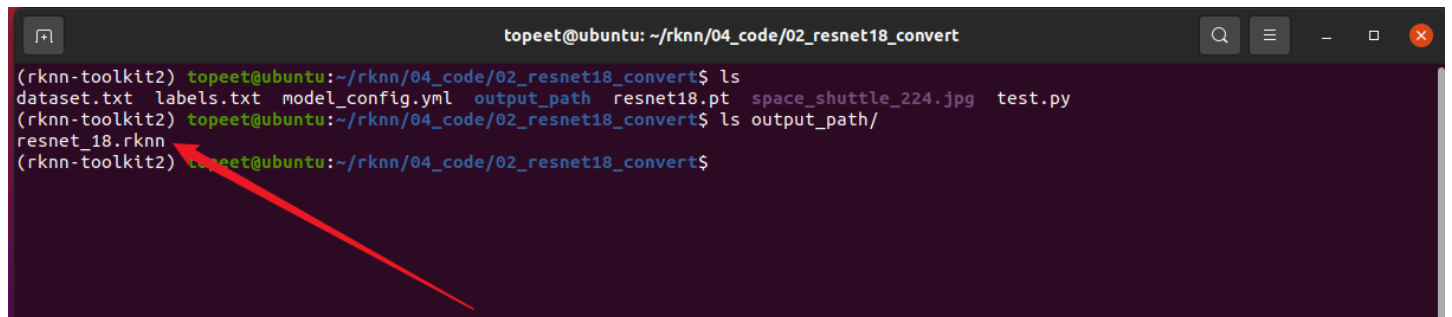
默认会出现报错，需要进行以下两个操作：

```
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/02_resnet18_convert$ python -m rknn.api.rknn_convert -t rk3588 -i ./model_config.yml -o
./output_path
Traceback (most recent call last):
  File "/home/topeet/miniconda3/envs/rknn-toolkit2/lib/python3.8/runpy.py", line 194, in _run_module_as_main
    return _run_code(code, main_globals, None,
  File "/home/topeet/miniconda3/envs/rknn-toolkit2/lib/python3.8/runpy.py", line 87, in _run_code
    exec(code, run_globals)
  File "/home/topeet/miniconda3/envs/rknn-toolkit2/lib/python3.8/site-packages/rknn/api/rknn_convert.py", line 9, in <module>
    import yaml
ModuleNotFoundError: No module named 'yaml'
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/02_resnet18_convert$
```

1.pip install pyyaml

2.把上一节的resnet18.pt模型复制到当前项目目录下

运行成功会如下图所示，可以看到rknn模型被转换出来了。



The terminal window shows the following commands and output:

```
topeet@ubuntu: ~/rknn/04_code/02_resnet18_convert
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/02_resnet18_convert$ ls
dataset.txt  labels.txt  model_config.yml  output_path  resnet18.pt  space_shuttle_224.jpg  test.py
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/02_resnet18_convert$ ls output_path/
resnet_18.rknn
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/02_resnet18_convert$
```

A red arrow points from the bottom left towards the file `resnet_18.rknn` in the `output_path/` directory, indicating the successful conversion of the model.